

Experiment 'p_6_10_same_set_seq' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 3

Hard fails (resolution): 1

Soft fails (resolution): 0

Verification attempts: 2

Problem Specification

Problem name: p_6_10_same_set_seq

Natural language statement: null

Method signature: p_6_10_same_set_seq(first: seq<int>, second: seq<int>) returns (are_same_set: bool)

Ensures

- are_same_set == (array_to_set(first) == array_to_set(second))

Functional Code Given

```
function array_to_set(arr: seq<int>): set<int>
{
  if |arr| == 0 then {}
  else {arr[0]} + array_to_set(arr[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_6_10_same_set_seq(first: seq<int>, second: seq<int>) returns (are_same_set: bool)`

The method should respect the following contract:

`ensures are_same_set == (array_to_set(first) == array_to_set(second))`

The contract uses the following dafny code:

```
function array_to_set(arr: seq<int>): set<int>
{
  if |arr| == 0 then {}
  else {arr[0]} + array_to_set(arr[1..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnOU4ggzV3669IvUTWdF7QkyAMZuM

Model: gpt-5.5-2026-04-23

Created at: 1780664560

Finish reason: stop

Usage: 410 tokens in, and 2105 tokens out

```
{
  are_same_set := forall x :: x in first <==> x in second;
}
```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

`problem_data/p_6_10_same_set_seq/program.dfy(10,18): Error: quantifiers in non-ghost`
→ `contexts must be compilable, but Dafny's heuristics can't figure out how to`
→ `produce or compile a bounded set of values for 'x'`

```

|
10 | are_same_set := forall x :: x in first <==> x in second;
| ~~~~~

```

1 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the

→ Dafny body, including the curly braces that surround it. Do not show the signature

→ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnOV2F80HL4OheTu2Dq6VPHleyu3o

Model: gpt-5.5-2026-04-23

Created at: 1780664620

Finish reason: stop

Usage: 587 tokens in, and 3674 tokens out

```

{
  are_same_set := (forall x | x in first :: x in second) &&
                  (forall x | x in second :: x in first);
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

problem_data/p_6_10_same_set_seq/program.dfy(9,0): Error: a postcondition could not be
→ proved on this return path

```

|
9 | {
| ^

```

problem_data/p_6_10_same_set_seq/program.dfy(8,22): Related location: this is the
→ postcondition that could not be proved

```

|
8 | ensures are_same_set == (array_to_set(first) == array_to_set(second))
| ^^

```

Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the

→ Dafny body, including the curly braces that surround it. Do not show the signature

→ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnOWrrKtVwf6t0Ikhpm0HOqcLXraB

Model: gpt-5.5-2026-04-23

Created at: 1780664733

Finish reason: stop

Usage: 852 tokens in, and 9046 tokens out

```
{
  var i := |first|;
  assert first[i..] == [];
  assert array_to_set(first[i..]) == {};
  assert forall x :: x in array_to_set(first[i..]) <==> x in first[i..];
  while i > 0
    invariant 0 <= i <= |first|
    invariant forall x :: x in array_to_set(first[i..]) <==> x in first[i..]
    decreases i
  {
    var j := i;
    i := i - 1;
    assert 0 <= i < |first|;
    assert j == i + 1;
    assert |first[i..]| > 0;
    assert first[i..][0] == first[i];
    assert first[i..][1..] == first[j..];
    assert first[i..] == [first[i]] + first[j..];
    assert array_to_set(first[i..]) == {first[i]} + array_to_set(first[j..]);
    assert forall x :: x in array_to_set(first[i..]) <==> x in first[i..] by {
      forall x
        ensures x in array_to_set(first[i..]) <==> x in first[i..]
      {
        assert x in array_to_set(first[j..]) <==> x in first[j..];
        assert x in array_to_set(first[i..]) <==> x in {first[i]} + array_to_set(first[j
          ↪ ..]);
        assert x in {first[i]} + array_to_set(first[j..]) <==> x == first[i] || x in
          ↪ array_to_set(first[j..]);
        assert x in first[i..] <==> x in [first[i]] + first[j..];
        assert x in [first[i]] + first[j..] <==> x == first[i] || x in first[j..];
      }
    }
  }
  assert i == 0;
  assert first[i..] == first;
  assert forall x :: x in array_to_set(first) <==> x in first;

  var k := |second|;
  assert second[k..] == [];
  assert array_to_set(second[k..]) == {};
  assert forall x :: x in array_to_set(second[k..]) <==> x in second[k..];
  while k > 0
    invariant 0 <= k <= |second|
    invariant forall x :: x in array_to_set(second[k..]) <==> x in second[k..]
    decreases k
  {
    var j := k;
```

```

k := k - 1;
assert 0 <= k < |second|;
assert j == k + 1;
assert |second[k..]| > 0;
assert second[k..][0] == second[k];
assert second[k..][1..] == second[j..];
assert second[k..] == [second[k]] + second[j..];
assert array_to_set(second[k..]) == {second[k]} + array_to_set(second[j..]);
assert forall x :: x in array_to_set(second[k..]) <==> x in second[k..] by {
  forall x
    ensures x in array_to_set(second[k..]) <==> x in second[k..]
  {
    assert x in array_to_set(second[j..]) <==> x in second[j..];
    assert x in array_to_set(second[k..]) <==> x in {second[k]} + array_to_set(second[
      ↪ j..]);
    assert x in {second[k]} + array_to_set(second[j..]) <==> x == second[k] || x in
      ↪ array_to_set(second[j..]);
    assert x in second[k..] <==> x in [second[k]] + second[j..];
    assert x in [second[k]] + second[j..] <==> x == second[k] || x in second[j..];
  }
}
}
assert k == 0;
assert second[k..] == second;
assert forall x :: x in array_to_set(second) <==> x in second;

are_same_set := (forall x | x in first :: x in second) &&
  (forall x | x in second :: x in first);

assert are_same_set ==> (forall x :: x in first <==> x in second) by {
  if are_same_set {
    assert forall x :: x in first <==> x in second by {
      forall x
        ensures x in first <==> x in second
      {
        if x in first {
          assert x in second;
        }
        if x in second {
          assert x in first;
        }
      }
    }
  }
}
assert (forall x :: x in first <==> x in second) ==> are_same_set by {
  if (forall x :: x in first <==> x in second) {
    assert (forall x | x in first :: x in second) by {
      forall x | x in first
        ensures x in second
      {

```

```

    assert x in first <==> x in second;
  }
}
assert (forall x | x in second :: x in first) by {
  forall x | x in second
    ensures x in first
  {
    assert x in first <==> x in second;
  }
}
}
}
assert are_same_set == (forall x :: x in first <==> x in second);

assert (forall x :: x in first <==> x in second) ==> array_to_set(first) ==
  ↪ array_to_set(second) by {
  if (forall x :: x in first <==> x in second) {
    assert forall x :: x in array_to_set(first) <==> x in array_to_set(second) by {
      forall x
        ensures x in array_to_set(first) <==> x in array_to_set(second)
      {
        assert x in array_to_set(first) <==> x in first;
        assert x in array_to_set(second) <==> x in second;
        assert x in first <==> x in second;
      }
    }
    assert array_to_set(first) == array_to_set(second);
  }
}
assert array_to_set(first) == array_to_set(second) ==> (forall x :: x in first <==> x
  ↪ in second) by {
  if array_to_set(first) == array_to_set(second) {
    assert forall x :: x in first <==> x in second by {
      forall x
        ensures x in first <==> x in second
      {
        assert x in array_to_set(first) <==> x in first;
        assert x in array_to_set(second) <==> x in second;
        assert x in array_to_set(first) <==> x in array_to_set(second);
      }
    }
  }
}
assert (array_to_set(first) == array_to_set(second)) == (forall x :: x in first <==> x
  ↪ in second);
}

```

Final Program

```
function array_to_set(arr: seq<int>): set<int>
```

```

{
  if |arr| == 0 then {}
  else {arr[0]} + array_to_set(arr[1..])
}

method p_6_10_same_set_seq(first: seq<int>, second: seq<int>) returns (are_same_set: bool
  ↪ )
  ensures are_same_set == (array_to_set(first) == array_to_set(second))
{
  var i := |first|;
  assert first[i..] == [];
  assert array_to_set(first[i..]) == {};
  assert forall x :: x in array_to_set(first[i..]) <==> x in first[i..];
  while i > 0
    invariant 0 <= i <= |first|
    invariant forall x :: x in array_to_set(first[i..]) <==> x in first[i..]
    decreases i
  {
    var j := i;
    i := i - 1;
    assert 0 <= i < |first|;
    assert j == i + 1;
    assert |first[i..]| > 0;
    assert first[i..][0] == first[i];
    assert first[i..][1..] == first[j..];
    assert first[i..] == [first[i]] + first[j..];
    assert array_to_set(first[i..]) == {first[i]} + array_to_set(first[j..]);
    assert forall x :: x in array_to_set(first[i..]) <==> x in first[i..] by {
      forall x
        ensures x in array_to_set(first[i..]) <==> x in first[i..]
      {
        assert x in array_to_set(first[j..]) <==> x in first[j..];
        assert x in array_to_set(first[i..]) <==> x in {first[i]} + array_to_set(first[j
          ↪ ..]);
        assert x in {first[i]} + array_to_set(first[j..]) <==> x == first[i] || x in
          ↪ array_to_set(first[j..]);
        assert x in first[i..] <==> x in [first[i]] + first[j..];
        assert x in [first[i]] + first[j..] <==> x == first[i] || x in first[j..];
      }
    }
  }
  assert i == 0;
  assert first[i..] == first;
  assert forall x :: x in array_to_set(first) <==> x in first;

  var k := |second|;
  assert second[k..] == [];
  assert array_to_set(second[k..]) == {};
  assert forall x :: x in array_to_set(second[k..]) <==> x in second[k..];
  while k > 0
    invariant 0 <= k <= |second|

```

```

invariant forall x :: x in array_to_set(second[k..]) <==> x in second[k..]
decreases k
{
  var j := k;
  k := k - 1;
  assert 0 <= k < |second|;
  assert j == k + 1;
  assert |second[k..]| > 0;
  assert second[k..][0] == second[k];
  assert second[k..][1..] == second[j..];
  assert second[k..] == [second[k]] + second[j..];
  assert array_to_set(second[k..]) == {second[k]} + array_to_set(second[j..]);
  assert forall x :: x in array_to_set(second[k..]) <==> x in second[k..] by {
    forall x
      ensures x in array_to_set(second[k..]) <==> x in second[k..]
    {
      assert x in array_to_set(second[j..]) <==> x in second[j..];
      assert x in array_to_set(second[k..]) <==> x in {second[k]} + array_to_set(second[
        ↪ j..]);
      assert x in {second[k]} + array_to_set(second[j..]) <==> x == second[k] || x in
        ↪ array_to_set(second[j..]);
      assert x in second[k..] <==> x in [second[k]] + second[j..];
      assert x in [second[k]] + second[j..] <==> x == second[k] || x in second[j..];
    }
  }
}
assert k == 0;
assert second[k..] == second;
assert forall x :: x in array_to_set(second) <==> x in second;

are_same_set := (forall x | x in first :: x in second) &&
  (forall x | x in second :: x in first);

assert are_same_set ==> (forall x :: x in first <==> x in second) by {
  if are_same_set {
    assert forall x :: x in first <==> x in second by {
      forall x
        ensures x in first <==> x in second
      {
        if x in first {
          assert x in second;
        }
        if x in second {
          assert x in first;
        }
      }
    }
  }
}
assert (forall x :: x in first <==> x in second) ==> are_same_set by {
  if (forall x :: x in first <==> x in second) {

```

```

    assert (forall x | x in first :: x in second) by {
      forall x | x in first
        ensures x in second
      {
        assert x in first <==> x in second;
      }
    }
    assert (forall x | x in second :: x in first) by {
      forall x | x in second
        ensures x in first
      {
        assert x in first <==> x in second;
      }
    }
  }
}
assert are_same_set == (forall x :: x in first <==> x in second);

assert (forall x :: x in first <==> x in second) ==> array_to_set(first) ==
  ↪ array_to_set(second) by {
  if (forall x :: x in first <==> x in second) {
    assert forall x :: x in array_to_set(first) <==> x in array_to_set(second) by {
      forall x
        ensures x in array_to_set(first) <==> x in array_to_set(second)
      {
        assert x in array_to_set(first) <==> x in first;
        assert x in array_to_set(second) <==> x in second;
        assert x in first <==> x in second;
      }
    }
    assert array_to_set(first) == array_to_set(second);
  }
}
assert array_to_set(first) == array_to_set(second) ==> (forall x :: x in first <==> x
  ↪ in second) by {
  if array_to_set(first) == array_to_set(second) {
    assert forall x :: x in first <==> x in second by {
      forall x
        ensures x in first <==> x in second
      {
        assert x in array_to_set(first) <==> x in first;
        assert x in array_to_set(second) <==> x in second;
        assert x in array_to_set(first) <==> x in array_to_set(second);
      }
    }
  }
}
assert (array_to_set(first) == array_to_set(second)) == (forall x :: x in first <==> x
  ↪ in second);
}

```

Total Token Usage

Input tokens: 1849

Output tokens: 14825

Reasoning tokens: 12955

Sum of ‘total tokens’: 16674

Experiment Timings

Overall Experiment started at 1780664558771, ended at 1780664914583, lasting 355812ms (355.81 seconds)

Iteration #1 started at 1780664558771, ended at 1780664619215, lasting 60444ms (60.44 seconds)

Iteration #2 started at 1780664619215, ended at 1780664733459, lasting 114244ms (114.24 seconds)

Iteration #3 started at 1780664733459, ended at 1780664914549, lasting 181090ms (181.09 seconds)